1.13. Feature Selection (Lựa chọn đặc trưng)

Các lớp trong module `sklearn.feature_selection` có thể được sử dụng để thực hiện **feature selection** (lựa chọn đặc trưng) hoặc **dimensionality reduction** (giảm chiều) trên tập dữ liệu.

Mục đích của việc này là:
- **Cải thiện độ chính xác của mô hình (estimators' accuracy scores)**.
- **Tăng hiệu suất** khi làm việc với các dataset có **số chiều rất lớn (high-dimensional datasets)**.

1.13.1 Removing Features with Low Variance

(Loại bỏ các đặc trưng có phương sai thấp)

`VarianceThreshold` là một phương pháp **baseline đơn giản** để lựa chọn đặc trưng.

Nguyên lý của nó:

- Loại bỏ các **feature có phương sai nhỏ hơn một ngưỡng (threshold)**.
- Mặc định, nó sẽ **loại bỏ tất cả các feature có phương sai bằng 0**.

Feature có **variance = 0** nghĩa là:
- Feature đó **có cùng giá trị ở tất cả các mẫu**.

Ví dụ:

Giả sử ta có một dataset với **các feature dạng boolean** (0 hoặc 1),
và ta muốn **loại bỏ các feature mà giá trị 0 hoặc 1 xuất hiện ở hơn 80% số mẫu**.

Các feature boolean có thể được xem như **biến ngẫu nhiên Bernoulli**.

Phương sai của biến Bernoulli được tính bằng:
$$
Var[X] = p(1 - p)
$$
Trong đó:

- \(p \) là xác suất biến nhận giá trị **1**
- \(1-p \) là xác suất biến nhận giá trị **0**

Chọn threshold

Nếu một feature có **80% giá trị giống nhau**:

$$
p = 0.8
$$

Khi đó:

$$
Var[X] = 0.8 \times (1 - 0.8)
$$

$$
Var[X] = 0.8 \times 0.2
$$

$$
Var[X] = 0.16
$$

Vì vậy ta có thể chọn:

`python
VarianceThreshold(threshold = 0.8 * (1 - 0.8))

In [1]:
```
from sklearn.feature_selection import VarianceThreshold
X = [[0, 0, 1], [0, 1, 0], [1, 0, 0], [0, 1, 1], [0, 1, 0], [0, 1, 1]]
sel = VarianceThreshold(threshold=(.8 * (1 - .8)))
sel.fit_transform(X)
```

Out[1]:
```
array([[0, 1],
       [1, 0],
       [0, 0],
       [1, 1],
       [1, 0],
       [1, 1]])
```

1.13.2 Univariate Feature Selection

(Lựa chọn đặc trưng đơn biến)

Univariate feature selection hoạt động bằng cách **chọn các feature tốt nhất dựa trên các kiểm định thống kê đơn biến (univariate statistical tests)**.

Phương pháp này có thể được xem như **một bước tiền xử lý (preprocessing)** trước khi đưa dữ liệu vào mô hình (estimator).

Trong Scikit-learn, các phương pháp feature selection được triển khai dưới dạng **object có phương thức** `transform` .

Các phương pháp phổ biến

1. SelectKBest

`SelectKBest` giữ lại **k feature có điểm số (score) cao nhất** và loại bỏ các feature còn lại.

Ví dụ:

`python
SelectKBest(score_func, k=10)

In [2]:
```
from sklearn.datasets import load_iris
from sklearn.feature_selection import SelectKBest
from sklearn.feature_selection import f_classif
X, y = load_iris(return_X_y=True)
print(X.shape)
X_new = SelectKBest(f_classif, k=2).fit_transform(X, y)
print(X_new.shape)
```

```
(150, 4)
(150, 2)
```

Scoring Functions trong Univariate Feature Selection

Các phương pháp feature selection này nhận đầu vào là một **hàm scoring**.
Hàm này trả về:

- **score đơn biến (univariate score)** cho mỗi feature
- **p-value** tương ứng

(Lưu ý: `SelectKBest` và `SelectPercentile` chỉ cần **score**, không cần p-value.)

Các hàm scoring phổ biến

Cho bài toán Regression

```
- r_regression
- f_regression
- mutual_info_regression
```

Cho bài toán Classification

```
- chi2
- f_classif
- mutual_info_classif
```

Ý nghĩa của các phương pháp

F-test methods

Các phương pháp dựa trên **F-test** (như `f_classif`, `f_regression`) dùng để ước lượng:

mức độ phụ thuộc tuyến tính (linear dependency) giữa hai biến ngẫu nhiên.

Nói đơn giản: feature ↑ → target ↑ hoặc ↓ theo quan hệ tuyến tính

Mutual Information

Các phương pháp **mutual information** có thể phát hiện:

- **mọi dạng phụ thuộc thống kê**
- không chỉ tuyến tính

Ví dụ có thể bắt được:

- quan hệ phi tuyến
- quan hệ phức tạp

Tuy nhiên vì là **non-parametric**, nên:

- cần **nhiều dữ liệu hơn** để ước lượng chính xác.

Chi-square test

`chi2` chỉ nên dùng với:

feature không âm (non-negative)

Ví dụ:

- tần suất từ (word frequency)
- số lần xuất hiện
- histogram

Không nên dùng với:

- dữ liệu có giá trị âm
- dữ liệu đã chuẩn hóa kiểu `StandardScaler`

Feature Selection với Sparse Data

Nếu bạn sử dụng **sparse data**
(tức dữ liệu biểu diễn dưới dạng **sparse matrix** như trong NLP):

Các hàm sau **xử lý trực tiếp sparse matrix** mà không cần chuyển sang dense:

- `chi2`
- `mutual_info_regression`
- `mutual_info_classif`

Điều này giúp:

- tiết kiệm RAM
- tăng tốc tính toán

⚠️ Warning

Không nên sử dụng **scoring function của regression cho bài toán classification**.

1.13.3 Recursive Feature Elimination (RFE)

Recursive Feature Elimination (RFE) là một phương pháp **feature selection dựa trên mô hình (model-based)**.

Ý tưởng:

Cho một **estimator bên ngoài (external estimator)** có khả năng gán **trọng số cho các feature**
ví dụ:

- hệ số của **Linear Model**
- `feature_importances_` của **tree-based models**

Mục tiêu của **RFE** là:

> Chọn các feature tốt nhất bằng cách **loại bỏ dần các feature ít quan trọng nhất**.

Cách RFE hoạt động

Quy trình của RFE:

1. Train mô hình trên **tập feature ban đầu**
2. Tính **độ quan trọng của từng feature**
3. **Loại bỏ feature ít quan trọng nhất**
4. Train lại mô hình với tập feature còn lại
5. Lặp lại quá trình trên

Quá trình này tiếp tục **cho đến khi đạt được số feature mong muốn**.

RFECV

`RFECV` là phiên bản nâng cao của **RFE**.

Khác biệt:

RFE → số feature phải chọn trước

RFECV → tự động tìm số feature tối ưu

RFECV sử dụng: cross-validation để tìm số lượng feature tốt nhất.

Cách RFECV hoạt động

Quy trình:

1. Thực hiện RFE trên các **cross-validation split**
2. Với mỗi số lượng feature:

- huấn luyện mô hình
- tính **evaluation score**

3. Tổng hợp kết quả trên các fold
4. Chọn **số feature tối đa hóa cross-validation score**

In [3]:
```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import RFECV
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold

# load data
X, y = load_breast_cancer(return_X_y=True)

model = LogisticRegression(max_iter=5000)

# cross validation
cv = StratifiedKFold(5)

selector = RFECV(
    estimator=model,
    step=1,
    cv=cv,
    scoring="accuracy"
)

selector.fit(X, y)

print("Optimal number of features:", selector.n_features_)
print("Selected features:", selector.support_)
```

```
Optimal number of features: 20
Selected features: [ True  True False False  True  True  True  True
True False  True  True
  True  True False False False False False False  True  True  True
False
  True  True  True  True  True  True]
```

In [4]:
```
X.shape
```

Out[4]: (569, 30)

1.13.4 Feature Selection using SelectFromModel

`SelectFromModel` là một **meta-transformer** dùng để thực hiện **feature selection dựa trên model**.

Nó có thể hoạt động với bất kỳ **estimator** nào có khả năng gán **độ quan trọng cho feature** thông qua:

- `coef_` (ví dụ: Linear Models)
- `feature_importances_` (ví dụ: Tree-based models)

Hoặc thông qua một **callable function** được cung cấp bởi tham số: `importance_getter`

Nguyên lý hoạt động

Sau khi model được **fit**, `SelectFromModel` sẽ:

1. Lấy **feature importance** từ model
2. So sánh với **threshold**
3. Feature nào có importance **nhỏ hơn threshold** sẽ bị loại bỏ

Threshold

Người dùng có thể chỉ định **threshold theo nhiều cách**:

1. Giá trị số

Ví dụ: threshold = 0.01, Feature có importance < 0.01 sẽ bị loại bỏ.

2. Heuristic có sẵn

Scikit-learn cung cấp một số heuristic: "mean", "median"

Ví dụ: threshold = "mean" → chỉ giữ feature có importance **lớn hơn mean importance**

---

3. Bội số của heuristic

Ví dụ:

threshold = "0.5mean"

threshold = "1.5median"

Giới hạn số feature

Có thể giới hạn số feature tối đa bằng tham số: max_features

Ví dụ: max_features = 10 → tối đa giữ lại **10 feature quan trọng nhất**

In [5]:
```
from sklearn.datasets import load_breast_cancer
from sklearn.feature_selection import SelectFromModel
from sklearn.linear_model import LogisticRegression

# load dataset
X, y = load_breast_cancer(return_X_y=True)

# model
model = LogisticRegression(max_iter=5000)

# feature selection
selector = SelectFromModel(model, threshold="mean")

# fit
X_new = selector.fit_transform(X, y)

print("Original shape:", X.shape)
print("New shape:", X_new.shape)
```

```
Original shape: (569, 30)
New shape: (569, 9)
```

In [6]:
```
from sklearn.datasets import load_breast_cancer
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel

# load data
X, y = load_breast_cancer(return_X_y=True)

model = RandomForestClassifier()

selector = SelectFromModel(
    model,
    threshold="median"
)

X_new = selector.fit_transform(X, y)

print("Original features:", X.shape[1])
print("Selected features:", X_new.shape[1])
```

```
Original features: 30
Selected features: 15
```

In [7]:
```
selector = SelectFromModel(
    RandomForestClassifier(),
    threshold="mean",
    max_features=10
)

X_new = selector.fit_transform(X, y)
X_new.shape
```

Out[7]:
```
(569, 8)
```

In [8]:
```
selector.get_support()
```

Out[8]:
```
array([False, False,  True, False, False, False,  True,  True, False,
       False, False, False, False,  True, False, False, False, False,
       False, False,  True, False,  True,  True, False, False, False,
        True, False, False])
```

1.13.5 Sequential Feature Selection

Sequential Feature Selection (SFS) có sẵn trong transformer `SequentialFeatureSelector` của `sklearn.feature_selection` .

SFS có thể hoạt động theo hai hướng:

- **Forward Selection (Forward-SFS)**
- **Backward Selection (Backward-SFS)**

Forward Sequential Feature Selection

Forward-SFS là một thuật toán **tham lam (greedy)** tìm kiếm tuần tự feature tốt nhất để thêm vào tập feature đã chọn.

Quy trình hoạt động:

1. Bắt đầu với **0 feature**.
2. Train model với **từng feature riêng lẻ**.
3. Chọn feature giúp **tối đa hóa cross-validation score**.
4. Thêm feature đó vào tập feature đã chọn.
5. Lặp lại quá trình:
- thử thêm từng feature còn lại
- chọn feature giúp tăng score tốt nhất
6. Dừng khi đạt **số lượng feature mong muốn** được xác định bởi tham số: `n_features_to_select`

Backward Sequential Feature Selection

Backward-SFS hoạt động theo hướng ngược lại:

1. Bắt đầu với **tất cả feature**.
2. Train model với toàn bộ feature.
3. Thử **loại bỏ từng feature** một.
4. Loại bỏ feature mà khi bỏ đi **làm giảm performance ít nhất** (hoặc làm tăng performance).
5. Lặp lại quá trình cho đến khi đạt số lượng feature mong muốn.

Tham số quan trọng

Tham số	Ý nghĩa
`n_features_to_select`	Số lượng feature cuối cùng muốn giữ
`direction`	`"forward"` hoặc `"backward"`
`estimator`	Model được dùng để đánh giá feature
`scoring`	Metric đánh giá (accuracy, f1, r2...)
`cv`	Cross validation

In [9]:
```
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

In [10]:
```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(max_iter=500)

sfs = SequentialFeatureSelector(
    model,
    n_features_to_select=5,
    direction="forward",
    scoring="accuracy",
    cv=5
)

sfs.fit(X_train, y_train)

selected_features = sfs.get_support()

print(selected_features)
```

```
[ True False False  True False False False False False False False
False
 False False False False False False False False False  True  True
False
 False  True False False False False]
```

In [11]:
```
import numpy as np

selected_index = np.where(sfs.get_support())[0]

print(selected_index)
```

```
[ 0  3 21 22 25]
```

Các phương pháp xử lý dữ liệu mất cân bằng

2.1.1 Naive Random Over-Sampling

Một cách để giải quyết vấn đề **mất cân bằng dữ liệu (imbalanced data)** là tạo thêm các mẫu dữ liệu mới cho những lớp **bị thiếu (under-represented classes)**.

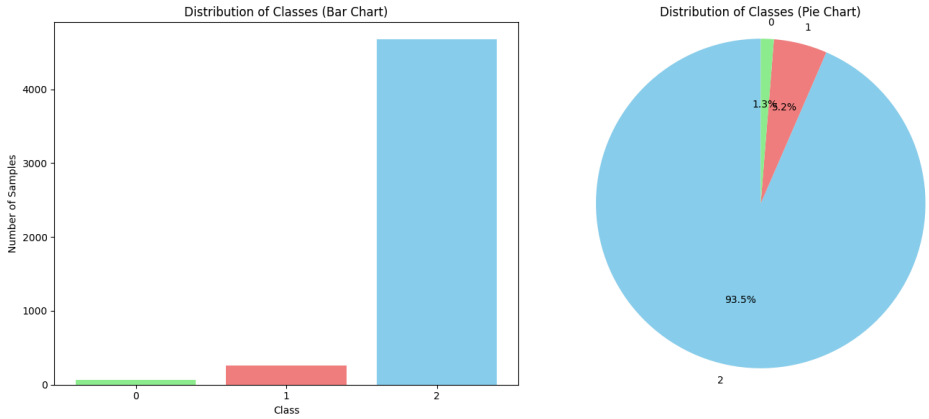
Chiến lược đơn giản nhất là **tạo các mẫu mới bằng cách lấy mẫu ngẫu nhiên có lặp lại (random sampling with replacement)** từ các mẫu hiện có.

Phương pháp này được gọi là **Naive Random Over-Sampling**.

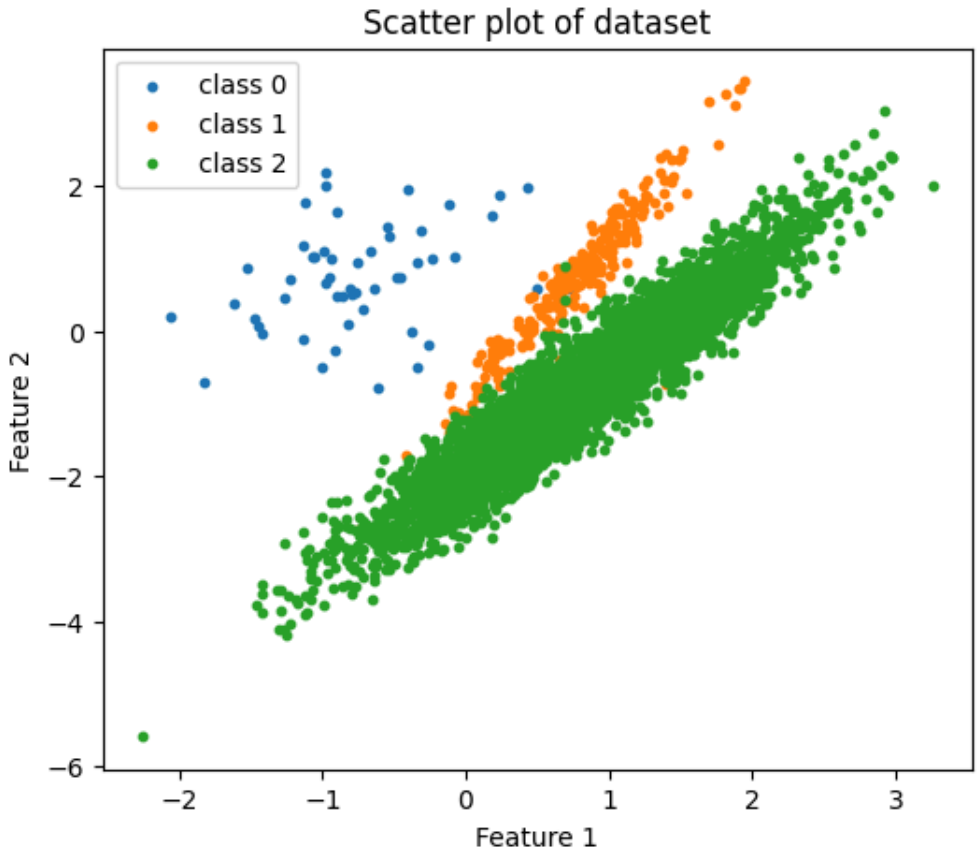
Trong thư viện `imbalanced-learn`, class `RandomOverSampler` cung cấp cơ chế thực hiện phương pháp này.

In [12]:
```
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=5000, n_features=2, n_informative=2,
                           n_redundant=0, n_repeated=0, n_classes=3,
                           n_clusters_per_class=1,
                           weights=[0.01, 0.05, 0.94],
                           class_sep=0.8, random_state=0)
```

In [13]:
```
import matplotlib.pyplot as plt
from collections import Counter

# Get class counts
class_counts = Counter(y)
labels = list(class_counts.keys())
sizes = list(class_counts.values())

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Bar chart
axes[0].bar(labels, sizes, color=['skyblue', 'lightcoral', 'lightgreen'])
axes[0].set_title('Distribution of Classes (Bar Chart)')
axes[0].set_xlabel('Class')
axes[0].set_ylabel('Number of Samples')
axes[0].set_xticks(labels)

# Pie chart
axes[1].pie(sizes, labels=labels, autopct='%1.1f%%', startangle=90, colors=
['skyblue', 'lightcoral', 'lightgreen'])
axes[1].set_title('Distribution of Classes (Pie Chart)')
axes[1].axis('equal')  # Equal aspect ratio ensures that pie is drawn as a
circle.

plt.tight_layout()
plt.show()
```



In [14]:
```
plt.figure(figsize=(6,5))

for label in np.unique(y):
    idx = y == label
    plt.scatter(X[idx,0], X[idx,1], label=f"class {label}", s=10)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Scatter plot of dataset")
plt.legend()
plt.show()
```



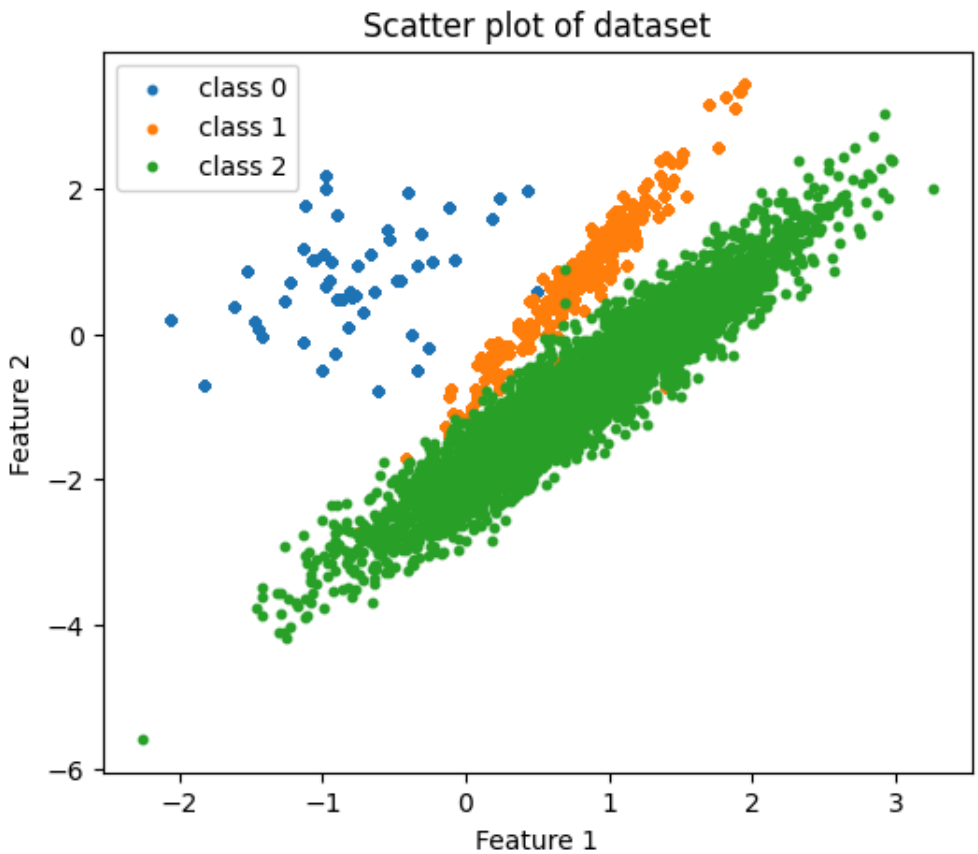
In [15]:
```
from collections import Counter
print(sorted(Counter(y).items()))
```

```
[(np.int64(0), 64), (np.int64(1), 262), (np.int64(2), 4674)]
```

In [16]:
```
from imblearn.over_sampling import RandomOverSampler
ros = RandomOverSampler(random_state=0)
X_resampled, y_resampled = ros.fit_resample(X, y)
print(sorted(Counter(y_resampled).items()))
```

```
[(np.int64(0), 4674), (np.int64(1), 4674), (np.int64(2), 4674)]
```

In [17]:
```
plt.figure(figsize=(6,5))

for label in np.unique(y_resampled):
    idx = y_resampled == label
    plt.scatter(X_resampled[idx,0], X_resampled[idx,1], label=f"class
{label}", s=10)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.title("Scatter plot of dataset")
plt.legend()
plt.show()
```



In [18]:
```
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    stratify=y,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\n===== BEFORE OVERSAMPLING =====")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

```
===== BEFORE OVERSAMPLING =====
Accuracy: 0.9853333333333333
              precision    recall  f1-score   support

           0       1.00      0.63      0.77        19
           1       0.96      0.82      0.88        79
           2       0.99      1.00      0.99      1402

    accuracy                           0.99      1500
   macro avg       0.98      0.82      0.88      1500
weighted avg       0.99      0.99      0.98      1500

Confusion Matrix
[[  12    2    5]
 [   0   65   14]
 [   0    1 1401]]
```

In [19]:
```
model_ros = LogisticRegression(max_iter=1000)

model_ros.fit(X_resampled, y_resampled)

y_pred_ros = model_ros.predict(X_test)

print("\n===== AFTER OVERSAMPLING =====")
print("Accuracy:", accuracy_score(y_test, y_pred_ros))
print(classification_report(y_test, y_pred_ros))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred_ros))
```

```
===== AFTER OVERSAMPLING =====
Accuracy: 0.9353333333333333
              precision    recall  f1-score   support

           0       0.20      0.74      0.31        19
           1       0.66      0.87      0.75        79
           2       1.00      0.94      0.97      1402

    accuracy                           0.94      1500
   macro avg       0.62      0.85      0.68      1500
weighted avg       0.97      0.94      0.95      1500

Confusion Matrix
[[  14    0    5]
 [   9   69    1]
 [  47   35 1320]]
```

In [20]:
```
model = LogisticRegression(max_iter=1000, class_weight='balanced')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\n===== BEFORE OVERSAMPLING =====")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

```
===== BEFORE OVERSAMPLING =====
Accuracy: 0.9406666666666667
              precision    recall  f1-score   support

           0       0.22      0.74      0.33        19
           1       0.69      0.89      0.77        79
           2       1.00      0.95      0.97      1402

    accuracy                           0.94      1500
   macro avg       0.63      0.86      0.69      1500
weighted avg       0.97      0.94      0.95      1500

Confusion Matrix
[[  14    0    5]
 [   8   70    1]
 [  43   32 1327]]
```

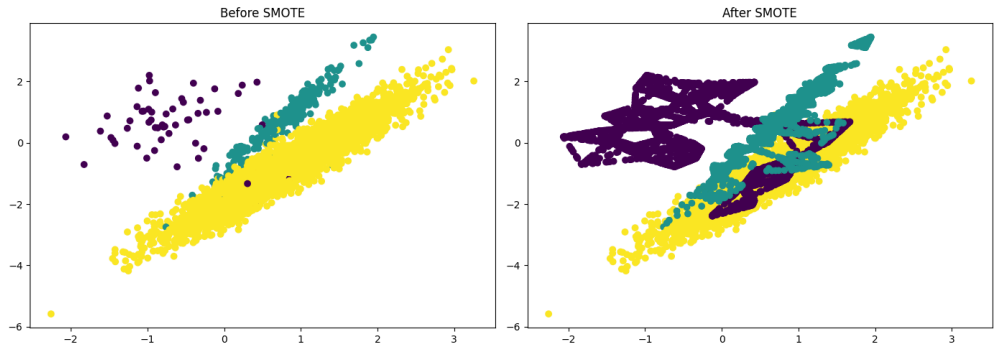
![image.png]
(

![image.png]
(

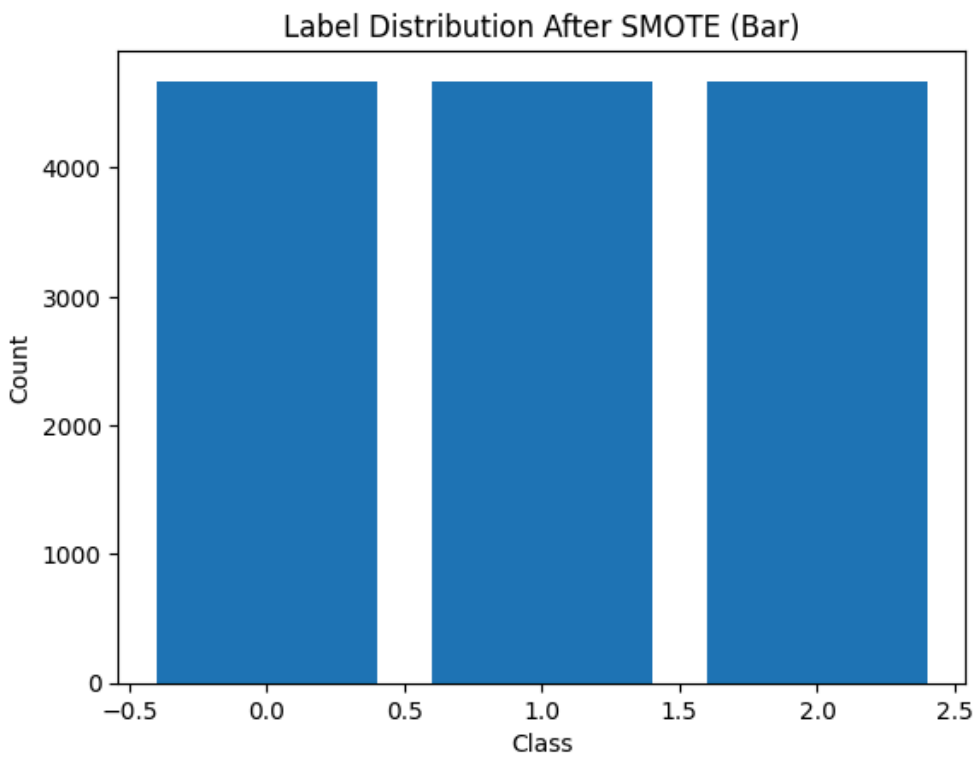
From random over-sampling to SMOTE and *ADASYN*

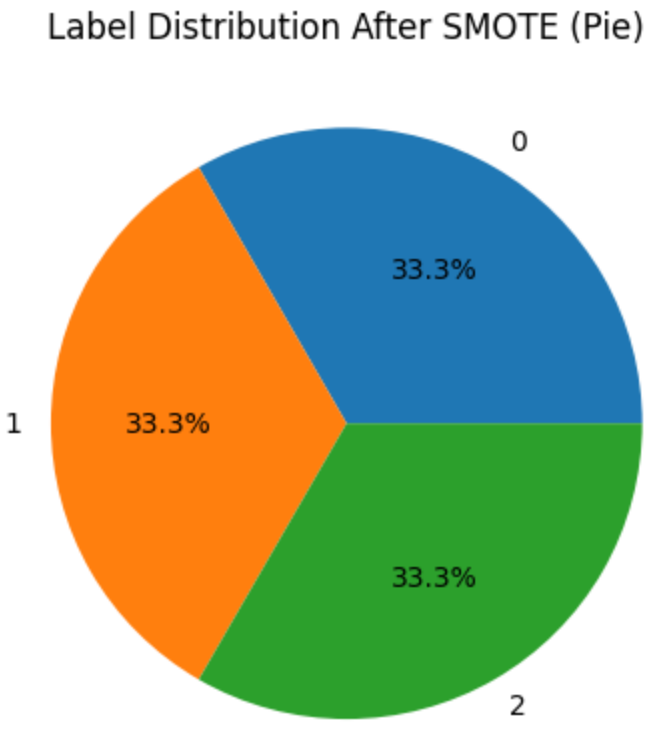
![image.png]
(

In [21]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)

# 2. Apply SMOTE
sm = SMOTE(random_state=0, k_neighbors = 5)
X_res, y_res = sm.fit_resample(X, y)

# 3. Scatter plot before & after SMOTE
plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before SMOTE")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After SMOTE")
plt.tight_layout()
plt.show()
```



In [22]:
```
unique, counts = np.unique(y_res, return_counts=True)

plt.figure()
plt.bar(unique, counts)
plt.title("Label Distribution After SMOTE (Bar)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()

# 5. Pie chart
plt.figure()
plt.pie(counts, labels=unique, autopct='%1.1f%%')
plt.title("Label Distribution After SMOTE (Pie)")
plt.show()
```




Label Distribution After SMOTE (Pie)
0
33.3%
1
33.3%
33.3%
2

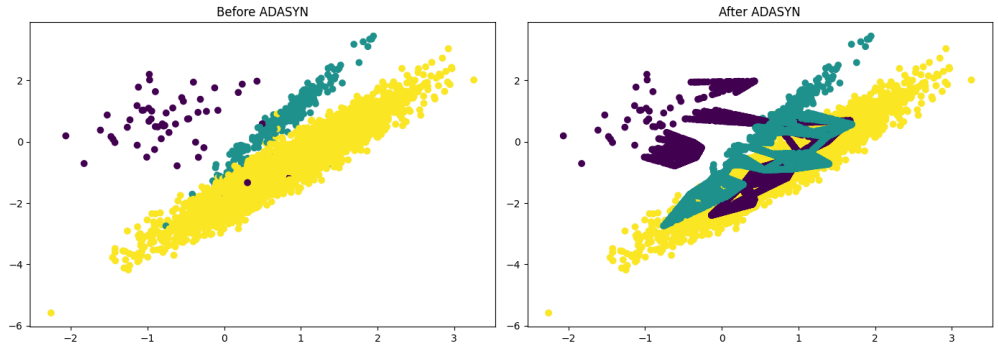
In [23]:
```
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\n===== BEFORE OVERSAMPLING =====")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

```
===== BEFORE OVERSAMPLING =====
Accuracy: 0.864036130259092
              precision    recall  f1-score   support

           0       0.88      0.78      0.83      1403
           1       0.93      0.88      0.91      1402
           2       0.80      0.93      0.86      1402

    accuracy                           0.86      4207
   macro avg       0.87      0.86      0.86      4207
weighted avg       0.87      0.86      0.86      4207

Confusion Matrix
[[1091   52  260]
 [  92 1237   73]
 [  57   38 1307]]
```

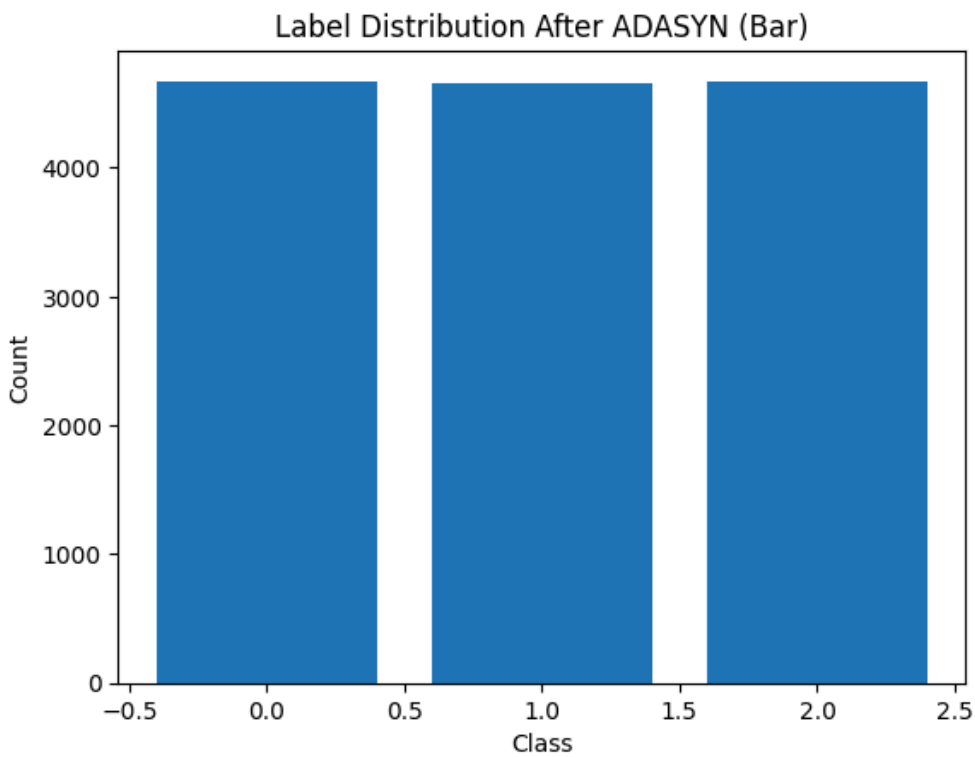
![image.png]
(

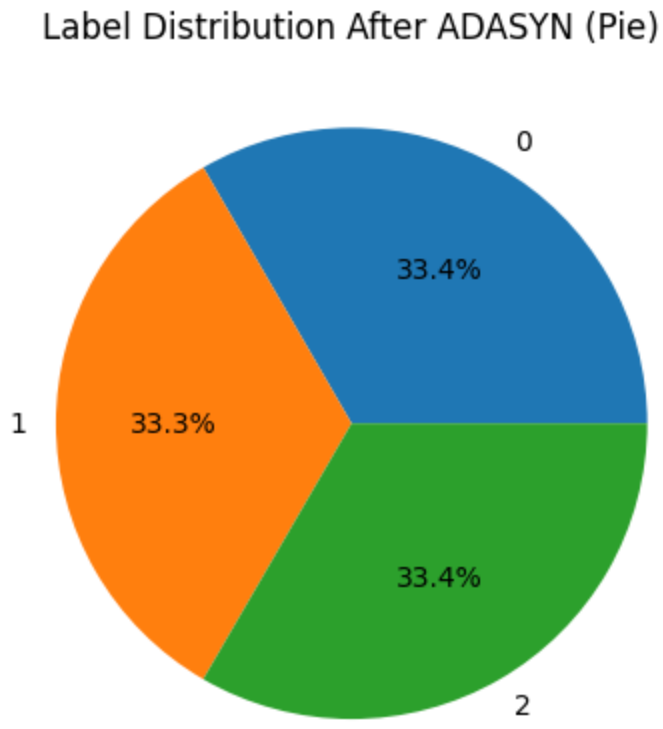
![image.png]
(

In [24]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE, ADASYN
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)


ada = ADASYN(random_state=0)
X_res, y_res = ada.fit_resample(X, y)

plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before ADASYN")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After ADASYN")
plt.tight_layout()
plt.show()
```



In [25]:
```
unique, counts = np.unique(y_res, return_counts=True)

plt.figure()
plt.bar(unique, counts)
plt.title("Label Distribution After ADASYN (Bar)")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()

# 5. Pie chart
plt.figure()
plt.pie(counts, labels=unique, autopct='%1.1f%%')
plt.title("Label Distribution After ADASYN (Pie)")
plt.show()
```




Label Distribution After ADASYN (Pie)
0
33.4%
1
33.3%
33.4%
2

In [26]:
```
from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\n===== BEFORE OVERSAMPLING =====")
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

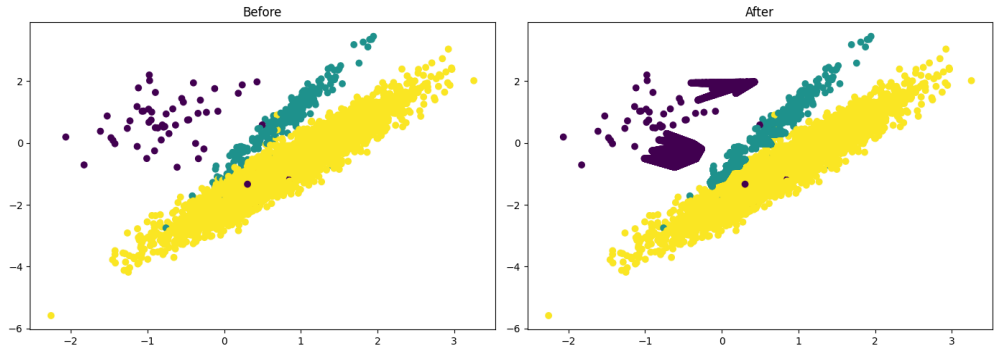
```
===== BEFORE OVERSAMPLING =====
Accuracy: 0.41660718534380203
              precision    recall  f1-score   support

           0       0.19      0.14      0.16      1402
           1       0.51      0.37      0.43      1399
           2       0.48      0.73      0.58      1402

    accuracy                           0.42      4203
   macro avg       0.40      0.42      0.39      4203
weighted avg       0.39      0.42      0.39      4203

Confusion Matrix
[[ 200  428  774]
 [ 550  522  327]
 [ 304   69 1029]]
```

![image.png]
(

![image.png]
(

In [27]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE, ADASYN, BorderlineSMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)


bl = BorderlineSMOTE(kind='borderline-1', random_state=0)
X_res, y_res = bl.fit_resample(X, y)

plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After")
plt.tight_layout()
plt.show()

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```

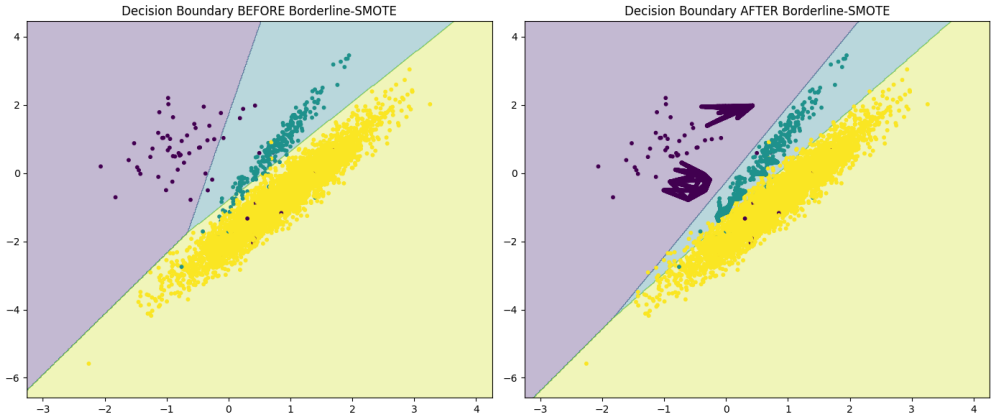



```
Accuracy: 0.9888281435702401
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1403
           1       0.97      1.00      0.98      1402
           2       0.99      0.97      0.98      1402

    accuracy                           0.99      4207
   macro avg       0.99      0.99      0.99      4207
weighted avg       0.99      0.99      0.99      4207

Confusion Matrix
[[1399    1    3]
 [   0 1397    5]
 [   1   37 1364]]
```

In [28]:
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import BorderlineSMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

# ========================
# 1. Generate Dataset
# ========================
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)

# ========================
# 2. Model BEFORE SMOTE
# ========================
model_before = LogisticRegression(max_iter=1000, multi_class="auto")
model_before.fit(X, y)

# ========================
# 3. Apply Borderline-SMOTE
# ========================
bl = BorderlineSMOTE(kind='borderline-1', random_state=0)
X_res, y_res = bl.fit_resample(X, y)

# ========================
# 4. Model AFTER SMOTE
# ========================
model_after = LogisticRegression(max_iter=1000, multi_class="auto")
model_after.fit(X_res, y_res)

# ========================
# 5. Tạo lưới để plot decision boundary
# ========================
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 500),
    np.linspace(y_min, y_max, 500)
)

# Predict trên lưới
Z_before = model_before.predict(np.c_[xx.ravel(), yy.ravel()])
Z_before = Z_before.reshape(xx.shape)

Z_after = model_after.predict(np.c_[xx.ravel(), yy.ravel()])
Z_after = Z_after.reshape(xx.shape)
```

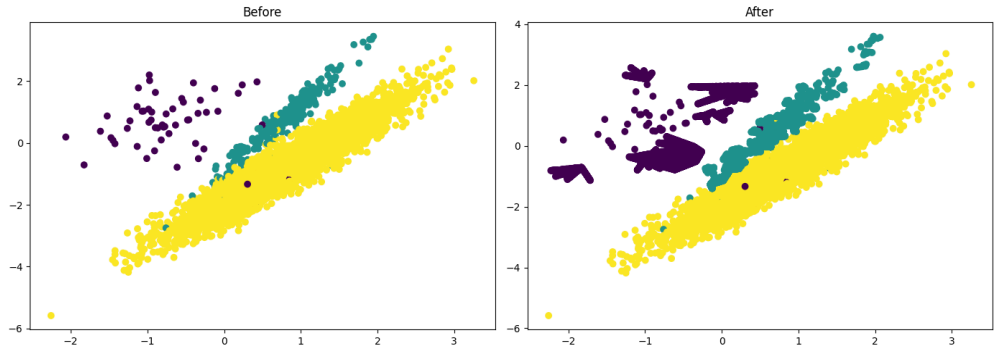
```
# ========================
# 6. Plot
# ========================
plt.figure(figsize=(14,6))

# BEFORE
plt.subplot(1,2,1)
plt.contourf(xx, yy, Z_before, alpha=0.3, cmap="viridis")
plt.scatter(X[:, 0], X[:, 1], c=y, cmap="viridis", s=10)
plt.title("Decision Boundary BEFORE Borderline-SMOTE")

# AFTER
plt.subplot(1,2,2)
plt.contourf(xx, yy, Z_after, alpha=0.3, cmap="viridis")
plt.scatter(X_res[:, 0], X_res[:, 1], c=y_res, cmap="viridis", s=10)
plt.title("Decision Boundary AFTER Borderline-SMOTE")

plt.tight_layout()
plt.show()
```

```
/usr/local/lib/python3.12/dist-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning:
'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to
its default value to avoid this warning.
  warnings.warn(
/usr/local/lib/python3.12/dist-
packages/sklearn/linear_model/_logistic.py:1247: FutureWarning:
'multi_class' was deprecated in version 1.5 and will be removed in
1.7. From then on, it will always use 'multinomial'. Leave it to
its default value to avoid this warning.
  warnings.warn(
```



In [29]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE, ADASYN, BorderlineSMOTE, SVMSMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)


svm_smote = SVMSMOTE(random_state=0)
X_res, y_res = svm_smote.fit_resample(X, y)

plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After")
plt.tight_layout()
plt.show()

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

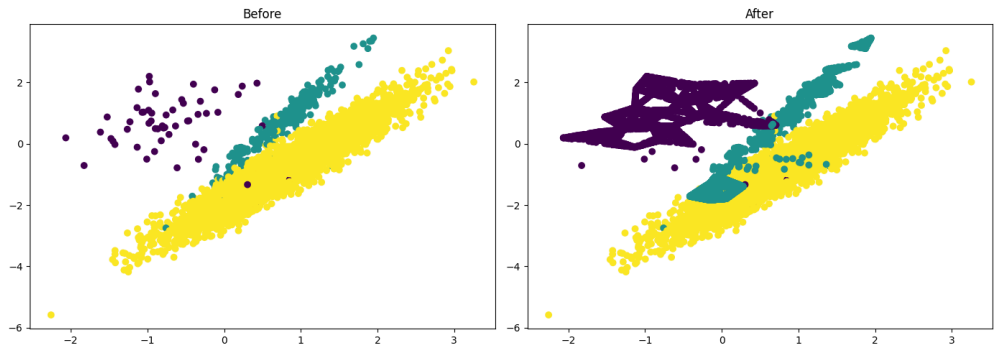
```
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```




```
Accuracy: 0.9897789398621345
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      1403
           1       0.98      1.00      0.99      1402
           2       0.99      0.98      0.98      1402

    accuracy                           0.99      4207
   macro avg       0.99      0.99      0.99      4207
weighted avg       0.99      0.99      0.99      4207

Confusion Matrix
[[1399    1    3]
 [   0 1397    5]
 [   2   32 1368]]
```

In [30]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE, ADASYN, BorderlineSMOTE,
SVMSMOTE, KMeansSMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)


kmeans_smote = KMeansSMOTE(
    k_neighbors=5,
    random_state=0,
    cluster_balance_threshold=0.01  # từ 0.5 xuống thấp
)
X_res, y_res = kmeans_smote.fit_resample(X, y)

plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After")
plt.tight_layout()
plt.show()

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```



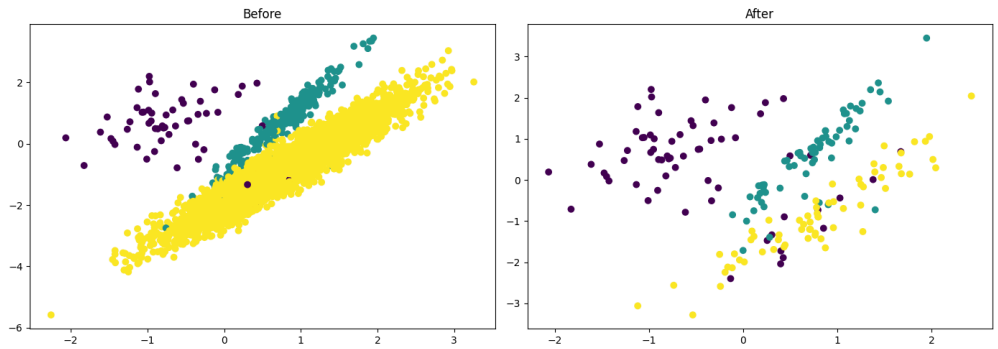

```
Accuracy: 0.9170627376425855
              precision    recall  f1-score   support

           0       1.00      0.98      0.99      1403
           1       0.87      0.89      0.88      1403
           2       0.89      0.88      0.88      1402

    accuracy                           0.92      4208
   macro avg       0.92      0.92      0.92      4208
weighted avg       0.92      0.92      0.92      4208

Confusion Matrix
[[1376   24    3]
 [   0 1251  152]
 [   0  170 1232]]
```

Các thuật toán để Under Sampling

In [31]:

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE, ADASYN, BorderlineSMOTE,
SVMSMOTE, KMeansSMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np
from imblearn.under_sampling import RandomUnderSampler

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)


rus = RandomUnderSampler(random_state=0)
X_res, y_res = rus.fit_resample(X, y)

plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After")
plt.tight_layout()
plt.show()

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
```

```
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```



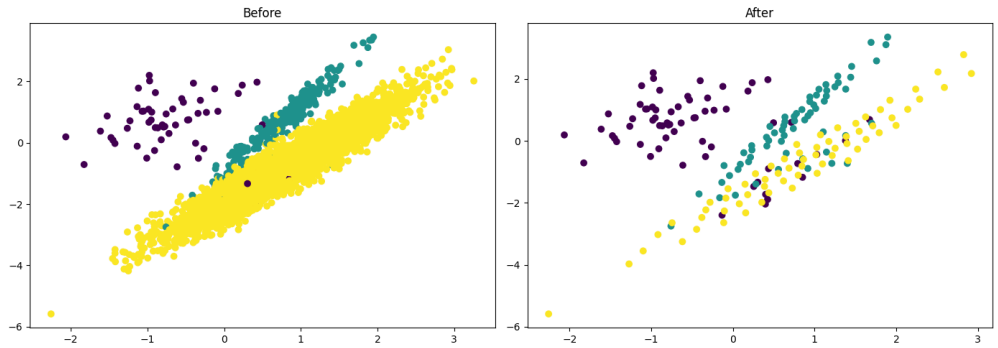

```
Accuracy: 0.8793103448275862
              precision    recall  f1-score   support

           0       0.89      0.80      0.84        20
           1       1.00      0.89      0.94        19
           2       0.78      0.95      0.86        19

    accuracy                           0.88        58
   macro avg       0.89      0.88      0.88        58
weighted avg       0.89      0.88      0.88        58

Confusion Matrix
[[16  0  4]
 [ 1 17  1]
 [ 1  0 18]]
```

![image.png]
(

![image.png]
(

In [32]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE, ADASYN, BorderlineSMOTE,
SVMSMOTE, KMeansSMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np
from imblearn.under_sampling import RandomUnderSampler

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)


from imblearn.under_sampling import ClusterCentroids
cc = ClusterCentroids(random_state=0)
X_res, y_res = cc.fit_resample(X, y)

plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After")
plt.tight_layout()
plt.show()

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```




```
Accuracy: 0.8275862068965517
              precision    recall  f1-score   support

           0       1.00      0.80      0.89        20
           1       0.88      0.79      0.83        19
           2       0.68      0.89      0.77        19

    accuracy                           0.83        58
   macro avg       0.85      0.83      0.83        58
weighted avg       0.86      0.83      0.83        58

Confusion Matrix
[[16  0  4]
 [ 0 15  4]
 [ 0  2 17]]
```

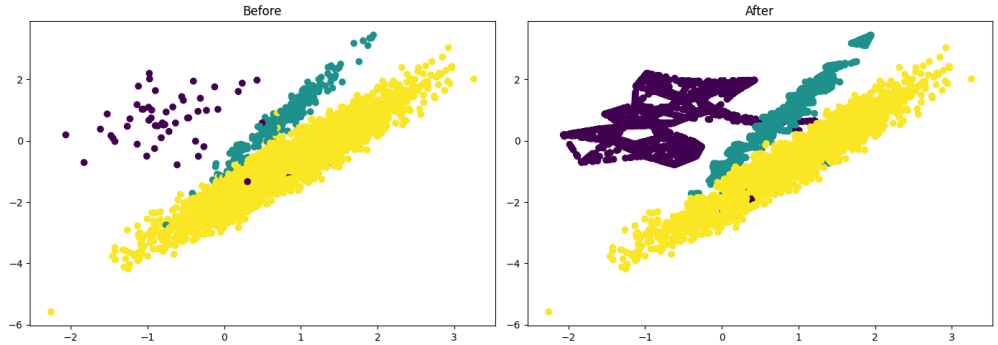
Phương pháp Hyprid

![image.png]
(

![image.png]
(

In [33]:
```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from imblearn.over_sampling import SMOTE, ADASYN, BorderlineSMOTE,
SVMSMOTE, KMeansSMOTE
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score, confusion_matrix
import numpy as np
from imblearn.under_sampling import RandomUnderSampler

# 1. Generate dataset
X, y = make_classification(
    n_samples=5000, n_features=2, n_informative=2,
    n_redundant=0, n_repeated=0, n_classes=3,
    n_clusters_per_class=1, weights=[0.01, 0.05, 0.94],
    class_sep=0.8, random_state=0
)

from imblearn.combine import SMOTEENN
smote_tomek = SMOTEENN(random_state=0)
X_res, y_res = smote_tomek.fit_resample(X, y)

plt.figure(figsize=(14,5))

plt.subplot(1,2,1)
plt.scatter(X[:,0], X[:,1], c=y)
plt.title("Before")

plt.subplot(1,2,2)
plt.scatter(X_res[:,0], X_res[:,1], c=y_res)
plt.title("After")
plt.tight_layout()
plt.show()

from sklearn.metrics import classification_report, accuracy_score,
confusion_matrix
X_train, X_test, y_train, y_test = train_test_split(
    X_res, y_res,
    test_size=0.3,
    stratify=y_res,
    random_state=0
)

# ============================
# 3. Logistic Regression (before oversampling)
# ============================
model = LogisticRegression(max_iter=1000)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print(classification_report(y_test, y_pred))
print("Confusion Matrix")
print(confusion_matrix(y_test, y_pred))
```




```
Accuracy: 0.9595311191738767
              precision    recall  f1-score   support

           0       0.98      0.91      0.95      1218
           1       0.97      0.99      0.98      1314
           2       0.92      0.97      0.95      1051

    accuracy                           0.96      3583
   macro avg       0.96      0.96      0.96      3583
weighted avg       0.96      0.96      0.96      3583

Confusion Matrix
[[1109   28   81]
 [   3 1305    6]
 [  15   12 1024]]
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.